Linux kernel rootkit detection challenges

It’s easy, too easy…

What we know to do well

1. Static/Code (comparison)
2. Disk vs Memory when possible
3. Check specifically in places in the kernel we know

There are 1000 more ways right??

Time to show that

```
procedure BuildTypeGraph()
 Nodes ← set of extracted types from kernel source
 Edges ← ∅
 FPNodes ← ∅
 for each Struct s ∈ Nodes
   do { for each member m ∈ Members(s)
        do { do { if IsFunctionPointer(m)
                  then FPNodes ← FPNodes ⋃ {s}
                  if IsStruct(m)
                  then Edges ← Edges ⋃ (s, Type(m)) }
 for each Struct n ∈ Nodes
   do { ExcludeNode ← true
        for each Struct f ∈ FPNodes
          do { do { if PathExists(Edges, n, f)
                    then { ExcludeNode ← false
                           break } }
        if ExcludeNode
          then RemoveNode(Nodes, Edges, n) }
 return (Nodes, Edges)
```

18,000 kernel function pointers in data areas

```
omershalev@ubuntu:/Users/omershalev/rnd/kernel-dev/kernel/linux-5.4.289$
$
$ ./enumerate_fp.sh
[+] Thinking....
[-] Skipping potential loop
Found 17,654 pointers in code
```

credit:
https://www.cs.umd.edu/~mwh/papers/sbcfi.pdf

I am not the only one - In Windows (Xp)

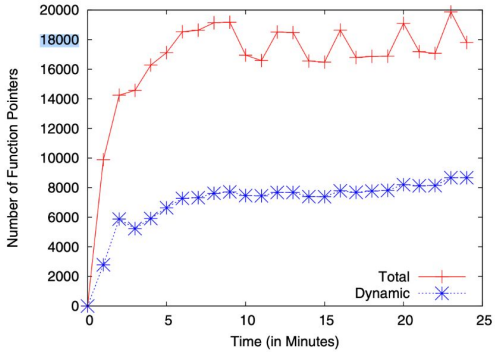


Fig. 4. Attack Space

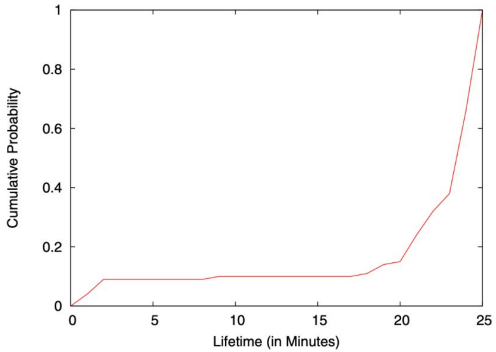
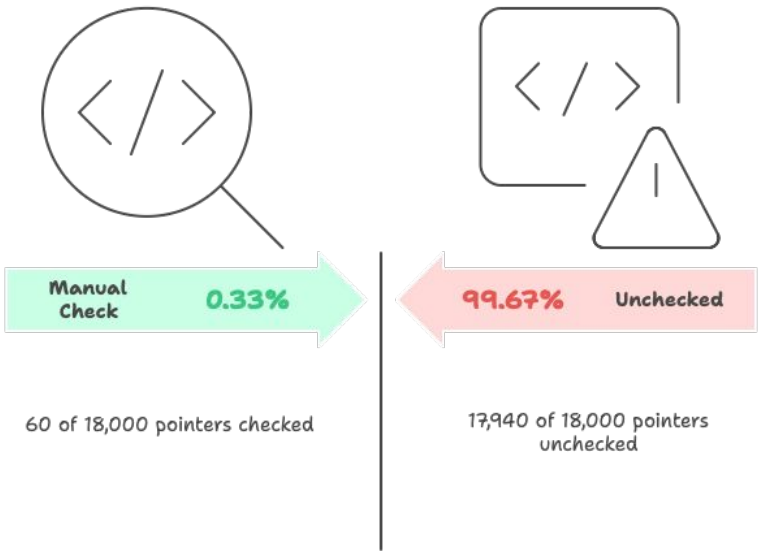


Fig. 5. Lifetime Distribution

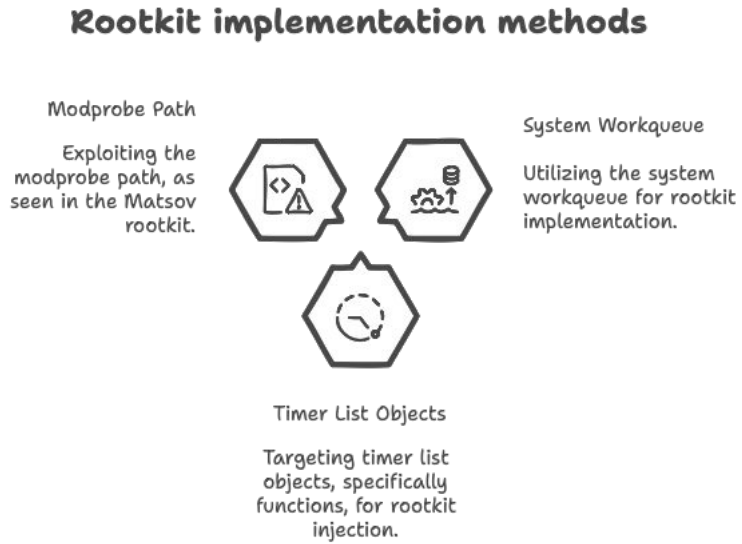
credit: http://bitblaze.cs.berkeley.edu/papers/hookscout-dimva10.pdf

Kernel Function Pointers

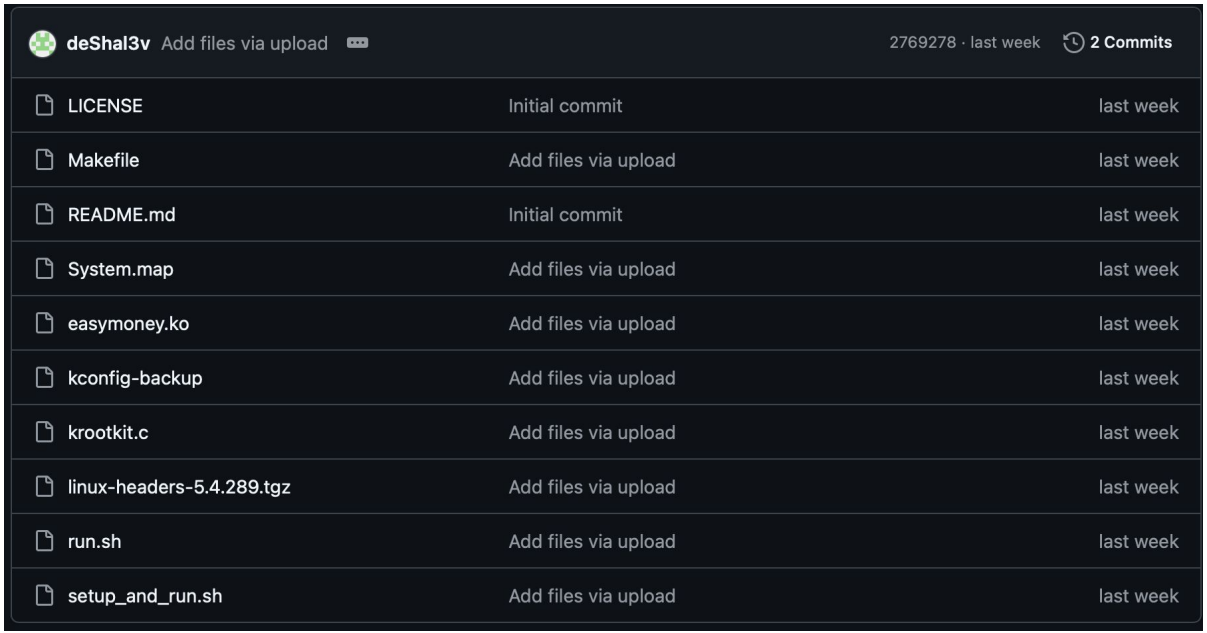


Small POC

Ex rookit for matsov



Small POC



deShal3v Add files via upload — 2769278 · last week — 2 Commits

LICENSE	Initial commit	last week
Makefile	Add files via upload	last week
README.md	Initial commit	last week
System.map	Add files via upload	last week
easymoney.ko	Add files via upload	last week
kconfig-backup	Add files via upload	last week
krootkit.c	Add files via upload	last week
linux-headers-5.4.289.tgz	Add files via upload	last week
run.sh	Add files via upload	last week
setup_and_run.sh	Add files via upload	last week

```
(gdb) set $timer_list_offset = (size_t)&((struct timer_list *)0)->entry
(gdb)
(gdb) set $node = $base->vectors[174].first
(gdb) p $node
$34 = (struct hlist_node *) 0xffff88807cb6c1f8
(gdb) set $timer_addr = (unsigned long)$node - $timer_list_offset
(gdb) set $timer = (struct timer_list *)$timer_addr
(gdb)
(gdb) p $timer
$35 = (struct timer_list *) 0xffff88807cb6c1f8
(gdb) printf "Timer: 0x%lx\n", $timer_addr
Timer: 0xffff88807cb6c1f8
(gdb) printf "  Function: 0x%lx (%ps)\n", $timer->function, $timer->function
  Function: 0xffffffff8107ca90 (0xffffffff8107ca90s)
(gdb) printf "  Expires: %lu (current jiffies: %lu)\n", $timer->expires, jiffies
  Expires: 4297358162 (current jiffies: 4297356417)
(gdb) printf "  Flags: 0x%lx\n", $timer->flags
  Flags: 0x2ba00000
(gdb) p 0xffffffff8107ca90
$36 = 18446744071579355792
(gdb) x/gx 0xffffffff8107ca90
0xffffffff8107ca90 <delayed_work_timer_fn>:     0xe0578d4828778b48
```

The problem of 7753 (not only)

1. Overfitting
2. DKOM: completely blind to unresearched mechanisms
3. Not enough resource to map significantly more kernel areas for research.

The Problem

Every Kernel mechanism requires dedicated ‘research’ that can be done by only few.

Not Doing research
not enough resources

Overfitting
ללכת בלי ולהרגיש עם

Kernel Rootkit Detection: Classification and Taxonomy

7753 knows to

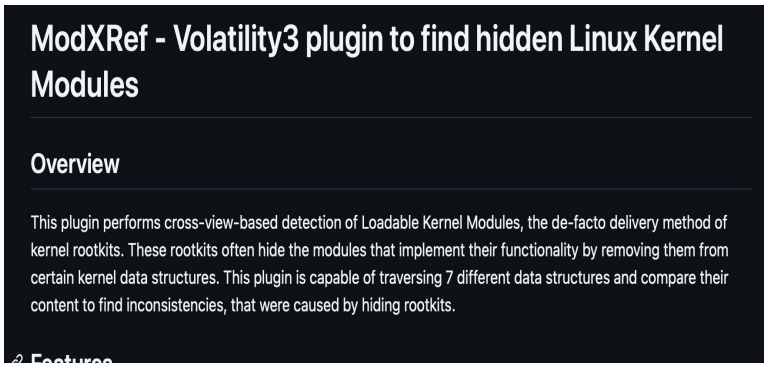
1. Signature-Based Detection
   - Identifies rootkits using known malware signatures, hash values, code patterns, or tampered kernel tables.
   - Fast, but easily evaded with code changes or advanced hiding.
2. Cross-View-Based Detection
   - Compares high-level OS outputs (e.g., process and module listings) with low-level memory scans to spot discrepancies from rootkit hiding methods (.
   - Often used in forensic analysis and live memory tools.
   - DKOM (60/18000)

Out of reach

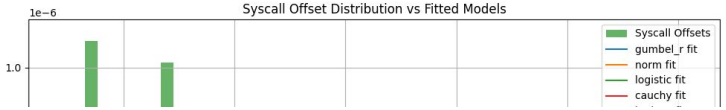
1. Behavior-Based Detection
   - Monitors kernel for suspicious activities: abnormal memory accesses, unauthorized function hooks, device driver anomalies, hidden files/processes, and direct data structure tampering.
   - Effective against stealth techniques, but can be noisy or subject to false positives.
2. Integrity-Based Detection
   - Checks and enforces kernel code/data integrity using hashes, write-protection, function pointer validation, and hardware policies.
   - Robust, but may introduce overhead and not cover all attack surfaces.
3. External Hardware-Based Detection
   - Uses coprocessors or isolated hardware modules to monitor kernel state and memory from outside the OS.
   - Highly resistant to tampering, but incurs extra cost and complexity.
4. Learning-Based Detection (AI/ML)
   - Utilizes machine and deep learning for profiling behavior, anomaly detection, and feature learning from kernel activity and memory/state data.
   - Adaptable to new threats, but depends on good data, features, and careful training.

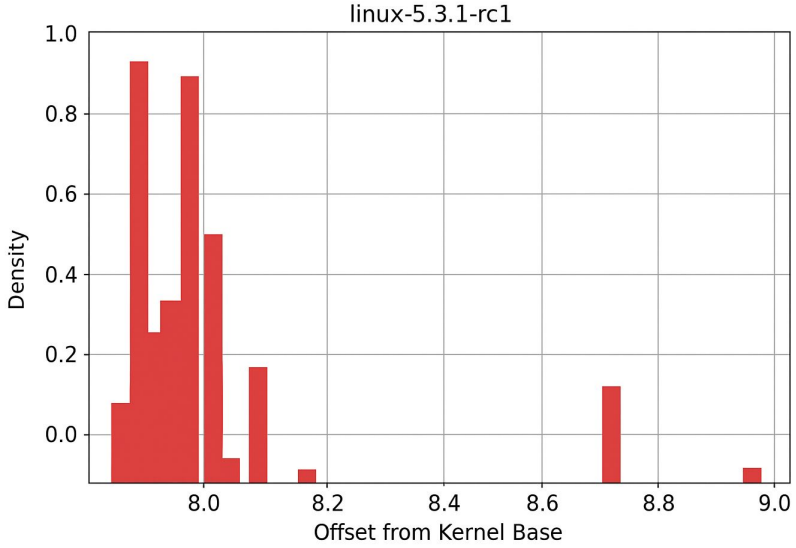
Areas we should get into

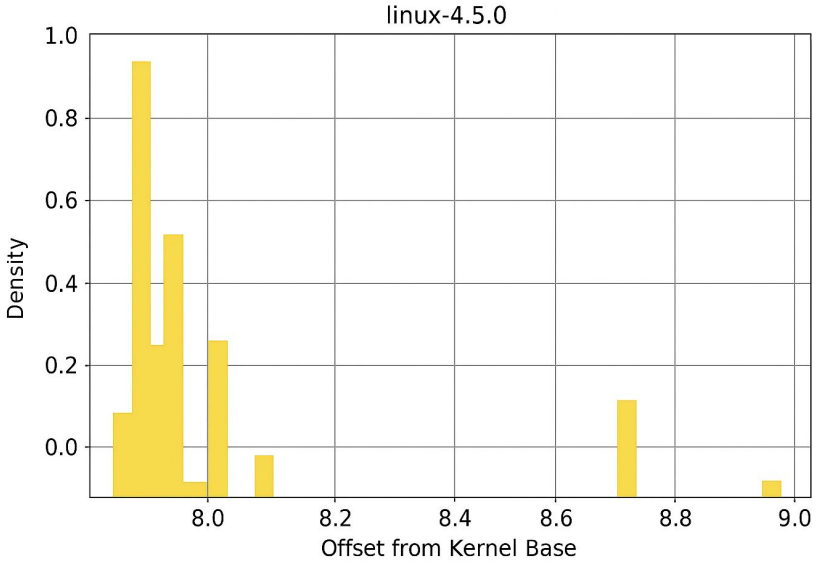
1. DKOM (direct kernel object manipulation) - finding a smarter approach
2. Emulation and Intervention (hypervisor based, emulation, eBPF)
3. Some outsource and open source:
    a. i.e (how to replace Seekle tomorrow)



Some experiments







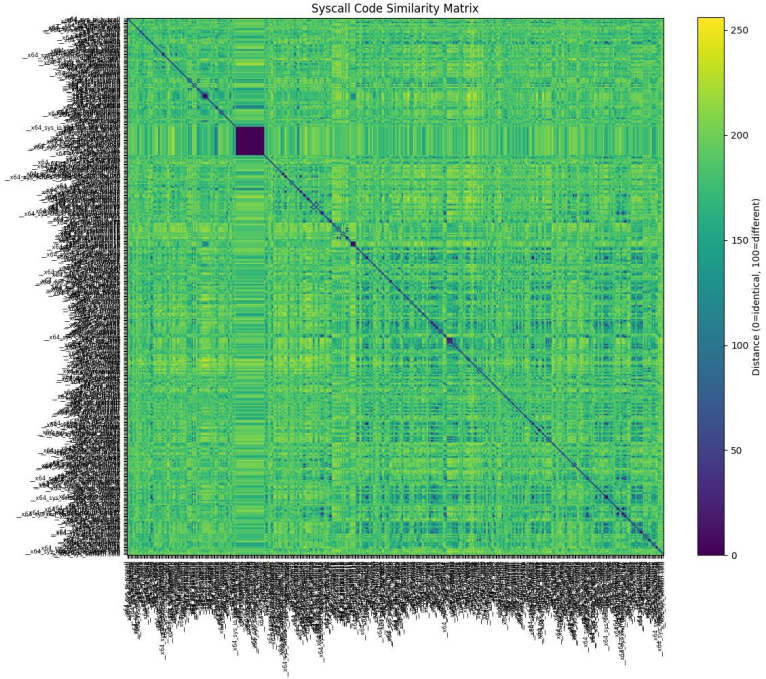
Some experiments

credit:
https://dl.ifip.org/db/conf/ifip11-9/df2007/WamplerG07.pdf

Distribution	AD Score
Largest Extreme Value	5.038
3-Parameter Gamma	6.617
3-Parameter Loglogistic	7.022
Logistic	7.026
Loglogistic	7.027
3-Parameter Lognormal	10.275
Lognormal	10.348
Normal	10.350
3-Parameter Weibull	49.346
Weibull	49.465
Smallest Extreme Value	49.471
2-Parameter Exponential	81.265
Exponential	116.956

Table 3. Rkit 1.01 results.

System	AD Score
Clean	5.038
Trojaned	99.210


Syscall Code Similarity Matrix
250
200
150
100
50
0
Distance (0=identical, 100=different)

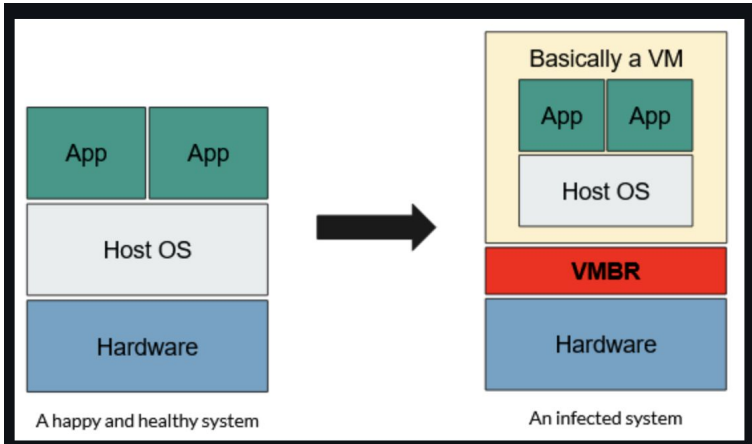
How will it look

```
$17 = (struct workqueue_struct *) 0xffff88807d416400
(gdb) p &system_wq
$18 = (struct workqueue_struct **) 0xffffffff8276fe90 <system_wq>
(gdb) p *system_wq
$19 = {pwqs = {next = 0xffff88807db30570, prev = 0xffff88807da30570}, list = {next = 0xffff88807d416610,
    prev = 0xffffffff82644110 <workqueues>}, mutex = {owner = {counter = 0}, wait_lock = {{rlock = {raw_lock = {{val = {
              counter = 0}, {locked = 0 '\000', pending = 0 '\000'}, {locked_pending = 0, tail = 0}}}}}}, osq = {
      tail = {counter = 0}}, wait_list = {next = 0xffff88807d416430, prev = 0xffff88807d416430}}, work_color = 0,
  flush_color = 0, nr_pwqs_to_flush = {counter = 0}, first_flusher = 0x0 <fixed_percpu_data>, flusher_queue = {
    next = 0xffff88807d416458, prev = 0xffff88807d416458}, flusher_overflow = {next = 0xffff88807d416468,
    prev = 0xffff88807d416468}, maydays = {next = 0xffff88807d416478, prev = 0xffff88807d416478},
  rescuer = 0x0 <fixed_percpu_data>, nr_drainers = 0, saved_max_active = 256, unbound_attrs = 0x0 <fixed_percpu_data>,
  dfl_pwq = 0x0 <fixed_percpu_data>, wq_dev = 0x0 <fixed_percpu_data>, name = "events", '\000' <repeats 17 times>, rcu = {
    next = 0x0 <fixed_percpu_data>, func = 0x0 <fixed_percpu_data>}, flags = 0, cpu_pwqs = 0x30500,
  numa_pwq_tbl = 0xffff88807d416510}
```

json

```
{
  "system_wq": {
    "lists": ["pwqs", "lists"],
    "known tampered": "rcu",
    "known fp": ""
  }
}
```

Motivation and tools



CoVirt (our virtual-machine based rootkit) can be simply thought of as "lifting" the rest of the kernel up, and "sliding underneath" to dynamically virtualize it. It basically asserts itself as the hypervisor for the running operating system and forces the rest of the kernel to run on top of it. Currently, once it establishes itself as the hypervisor, CoVirt is able to perform a couple "malicious" services (just basic proof-of-concepts).

Debug Registers

DEBUG REGISTERS

31 … 0	
LINEAR BREAKPOINT ADDRESS 0	DR0
LINEAR BREAKPOINT ADDRESS 1	DR1
LINEAR BREAKPOINT ADDRESS 2	DR2
LINEAR BREAKPOINT ADDRESS 3	DR3
Intel reserved. Do not define.	DR4
Intel reserved. Do not define.	DR5
BREAKPOINT STATUS	DR6
BREAKPOINT CONTROL	DR7

Action Items

complete research - learning ba

1. call stack modelling using ePBF trace demo an
learning based method

2. smart kernel specific da ith user
input

36h PoC for PoC at hom
each

complete reaserach - monitor based

ine lifting - both amd and intel and monitoring

registers access monitoring

5. eBPF memory inspection in realtime

6. emulation based for kernel (qemu/using kexec
maybe)

48h PoC at home:

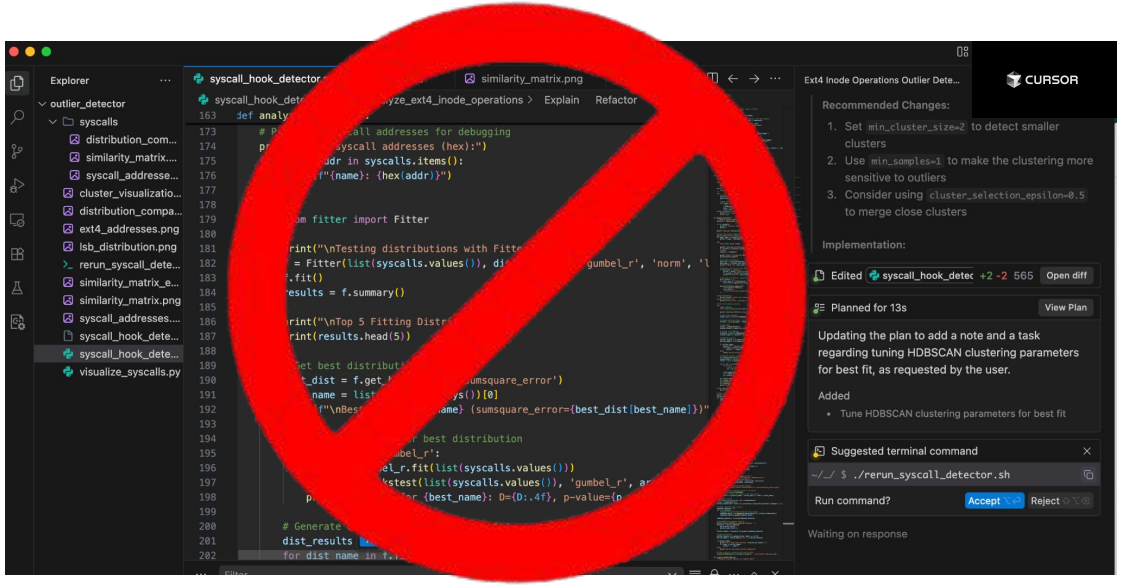
1. find best lead
2. implement it

What we have done already

1. Market research
   a. Knows exactly what exists out th so much in companies IP)
   b. Read ~15 research papers fro
2. Putting the finger on the problem.
3. Demonstrating two kernel rootkits using this (that we didnt find…)
4. Starting to play with the data and finding stuff. H lls, ext4_operations.
5. Writing eBPF probing mechanism to take kernel trac ost done

Challenges for me (i.e Help me to help you)

1.



Challenges for me (i.e Help me to help you)

2.

Opportunities (Why Now)

1. Semi free - looking for new challenge after Hexagon
2. The rise and availability of llms/statistical learning frameworks and tools
    a. **Got the best training in the IDF - let me use this….**
    b. **Academic background**
3. **The new ai tools let me cover up on my weaknesses that usually hold me back**
    a. I am very slow - windsurf and cursor help a lot to mitigate and make it less noticeable in my work
    b. No need to do dev alone anymore - new llms are great. **By choosing and training the right ones you can get very good results for linux based coding.**

Ideas - what can we do

1. Work from home some of the day - i.e arriving late and utilizing the morning.
2. Work from other facility
3. Keep the spark - hackathon based and freedom.